

Algebraic Data Types in Scala

Do not use variables, side effects, exceptions or return statements (unless explicitly asked for). Do use the automatic tests and compiler errors to see whether you are doing fine. No hand-in this week.

Exercise 1 [E]. What is the value of the following match expression?¹ Answer without running the code.

```
import adpro.adt.List.*
List(1, 2, 3, 4, 5) match
  case Cons(x, Cons(2, Cons(4, _))) => x
  case Nil => 42
  case Cons(x, Cons(y, Cons(3, Cons(4, _)))) => x + y
  case Cons(h, t) => h
  case _ => 101
```

Which function from `adpro.adt` is called in this example?

Remark. For pedagogical reasons, all exercises below use our own implementation of lists, not the Scala standard library. Do *not* use online API docs to find the available functions, as they will be different. Find our implementation in `Exercises.scala`, at the very top. This is the only API that is available. Additionally, you can use the functions from earlier exercises to solve the later ones.

Exercise 2 [E]. Implement function `tail` for removing the first element of a list. The function should run in constant time. Throw the `NoSuchElementException` exception if given an empty list.²

```
def tail[A](as: List[A]): List[A]
```

Hint. The constant running time indicates that we are not copying the list. Otherwise the entire list would have to be traversed and we would have used linear time.

Exercise 3 [E]. Generalize `tail` to `drop`, a function that removes the first `n` elements from a list. The running time should be proportional to `n`—no need to make a copy of the entire list. Throw `NoSuchElementException` if the list is too short.³ For non-positive `n` the list is unchanged.

```
def drop[A](l: List[A], n: Int): List[A]
```

Exercise 4 [M]. Implement `dropWhile`, which removes elements starting the head of the list `l`, as long as they satisfy a predicate `p`. Do not use exceptions: if all elements satisfy `p` then return the empty list.⁴

```
def dropWhile[A](l: List[A], p: A => Boolean): List[A]
```

Remark. `dropWhile` is useful when we process a list and search only for interesting elements. We can then characterize what interests us as not `p` and ignore elements until we meet what we want. Often, however, it is more elegant to just filter the elements satisfying `p`.

Exercise 5 [H]. Implement a function `init` that returns a list consisting of all but the last element of the original list. Given `List(1, 2, 3, 4)`, the function returns `List(1, 2, 3)`. Throw `NoSuchElementException` if the list is empty.

¹Exercise 3.1 [Pilquist, Chiusano, Bjarnason, 2023]

²Exercise 3.2 [Pilquist, Chiusano, Bjarnason, 2023]

³Exercise 3.4 [Pilquist, Chiusano, Bjarnason, 2023]

⁴Exercise 3.5 [Pilquist, Chiusano, Bjarnason, 2023]

```
def init[A](l: List[A]): List[A]
```

Does this function take constant time, like `tail`? Does it take constant space?⁵

Exercise 6 [M]. Compute the length of a list using `foldRight`.⁶ Remember that `foldRight` has been presented briefly in the lecture slides, in the text book; you can find it in the top of the file `Exercises.scala`. Also, the next exercise has an example demonstrating the essence of `foldRight`.

```
def length[A](l: List[A]): Int
```

Remark. `foldLeft` and `foldRight` replace `for` loops from imperative programming. They are more elegant than direct recursion, but still fairly low-level. We use them if the standard operations (like `map`, `filter`, `zip`, etc.) do not suffice. See below.

Exercise 7 [M]. The function `foldRight` presented in the book is not tail-recursive and will result in a `StackOverflowError` for large lists. Convince yourself that this is the case, and then write another general list-recursion function, `foldLeft`, that *is* tail-recursive:

```
def foldLeft[A,B](l: List[A], z: B) (f: (B, A) => B): B
```

For comparison consider that:

`foldLeft(List(1, 2, 3, 4), 0) (_ + _)` computes $((0 + 1) + 2) + 3 + 4$ while
`foldRight(List(1, 2, 3, 4), 0) (_ + _)` computes $1 + (2 + (3 + (4 + 0)))$.

In this case the result is obviously the same, but not always so.⁷

Exercise 8 [M]. Write `product` (computing a product of a list of integers) and a function to compute the length of a list using `foldLeft`.⁸

Exercise 9 [H]. Write a function that returns the reverse of a list (given `List (1,2,3)`, it returns `List (3,2,1)`). Use one of the fold functions.⁹

Exercise 10 [M]. Write `foldRight` using `foldLeft` and `reverse`. The left fold performs the dual operation to the right one, so if you reverse the list you should be able to simulate one with the other. This version of `foldRight` is useful because it is tail-recursive, which means it works even for large lists without overflowing the stack. On the other hand, it is slower by a constant factor.

Exercise 11 [H]. Write `foldLeft` in terms of `foldRight`. Do not use `reverse` here (`reverse` is a special case of `foldLeft` so a solution based on `reverse` is cheating).

Hint: This may well be the most difficult exercise in the entire course. Synthesize a function that computes the run of `foldLeft`, and then invoke this function. To implement `foldLeft[A, B]` you will be calling `foldRight` with the following type parameters:

```
foldRight[A, B => B] (... , ..., ...)
```

This will compute a new function, which then needs to be called.¹⁰

⁵Exercise 3.6 [Pilquist, Chiusano, Bjarnason, 2023]

⁶Exercise 3.9 [Pilquist, Chiusano, Bjarnason, 2023]

⁷Exercise 3.10 [Pilquist, Chiusano, Bjarnason, 2023]

⁸Exercise 3.11 [Pilquist, Chiusano, Bjarnason, 2023]

⁹Exercise 3.12 [Pilquist, Chiusano, Bjarnason, 2023]

¹⁰Exercise 3.13 [Pilquist, Chiusano, Bjarnason, 2023]

Note: From now on, the use of explicit recursion is bad-smell for us. Only use explicit recursion when dealing with a non-standard iteration. Otherwise, you should use a suitable HOF. Similarly, a you should only use fold if any of the other simpler HOFs cannot.

Exercise 12[M]. Write a function that concatenates a list of lists into a single list. Its runtime should be linear in the total length of all lists. Use append that concatenates two lists (find it in the book and in the source file).¹¹

Exercise 13[M]. Implement filter that removes from a list the elements that do not satisfy p.¹²

```
def filter[A] (l: List[A]) (p: A => Boolean): List[A]
```

Exercise 14[M]. Write a function flatMap that works like map except that f, the function mapped, returns a list instead of a single value, and the result is automatically flattened to a list like with concat:

```
def flatMap[A,B] (l: List[A]) (f: A => List[B]): List[B]
```

For instance, flatMap(List(1, 2, 3)) (i => List(i, i)) results in List(1, 1, 2, 2, 3, 3). Together with map, (flatMap) will be key in the rest of the course. Understand this well.¹³

Exercise 15[M]. Use flatMap to re-implement filter.¹⁴

Exercise 16[M]. Write a recursive function that accepts two lists of integers and constructs a new list by adding elements at the same positions. If the lists are not of the same length, the function drops trailing elements of either list. For example, the lists List(1,2,3) and List(4,5,6,7) become List(5,7,9).¹⁵

Exercise 17[M]. Generalize the function you just wrote so that it is not specific to integers or addition. It should work with arbitrary binary operations. Name the new function zipWith.¹⁶

Exercise 18[H]. Implement a function hasSubsequence for checking whether a List contains another List as a subsequence. For instance, List(1,2,3,4) would have List(1,2), List(2,3), and List(4) as subsequences, among others, but List(1,4) is not its subsequence. You may have some difficulty finding a concise purely functional implementation that is also efficient. That is okay. Implement the function that comes most naturally, but is not necessarily efficient. Note: Any two values x and y can be compared for equality in Scala using the expression x == y. Here is the suggested type:

```
def hasSubsequence[A] (sup: List[A], sub: List[A]): Boolean
```

Recall that an empty sequence is a subsequence of any other sequence.¹⁷

¹¹Exercise 3.15 [Pilquist, Chiusano, Bjarnason, 2023]

¹²Exercise 3.19 [Pilquist, Chiusano, Bjarnason, 2023]

¹³Exercise 3.20 [Pilquist, Chiusano, Bjarnason, 2023]

¹⁴Exercise 3.21 [Pilquist, Chiusano, Bjarnason, 2023]

¹⁵Exercise 3.22 [Pilquist, Chiusano, Bjarnason, 2023]

¹⁶Exercise 3.23 [Pilquist, Chiusano, Bjarnason, 2023]

¹⁷Exercise 3.24 [Pilquist, Chiusano, Bjarnason, 2023]